

Krefasy Backoffice — Guia Técnico para Desenvolvedores

Versão: 1.0
Data: Julho 2026
Público: Novos desenvolvedores da equipa

1. Visão geral

O **Krefasy Backoffice** é uma aplicação web administrativa para gestão de créditos e empréstimos. Permite que administradores gerem clientes, empréstimos, parcelas, configurações de produtos financeiros e atendimento por chat.

Item	Detalhe
Nome do projeto	backoffice-krefasy--app
Template base	Vristo Vue (customizado para Krefasy)
API backend	https://credit-api.krefasy.com/api/v1
Acesso	Apenas utilizadores com role Admin
Repositório	Monorepo local em Palata App/backoffice-krefasy--app

2. Stack tecnológica

Frontend

Tecnologia	Versão	Uso
Vue 3	3.5+	Framework UI (Composition API + <script setup>)
TypeScript	5.5+	Tipagem estática
Vite	5.4+	Bundler e dev server
Vue Router 4	4.x	Routing com lazy loading
Pinia	2.x	Gestão de estado
Tailwind CSS	3.4+	Estilização utilitária
Axios	1.x	Cliente HTTP para API
ApexCharts	3.x	Gráficos no dashboard
SweetAlert2	11.x	Diálogos de confirmação/erro
vue-i18n	9.x	Internacionalização (PT/EN e outros)

Ferramentas de desenvolvimento

- `vue-tsc` — verificação de tipos TypeScript
- `yarn` — gestor de pacotes (preferido no projeto)
- Docker — deploy em contêiner (`Dockerfile` incluído)

3. Estrutura de pastas

```
backoffice-krefasy--app/
├─ public/                # Assets estáticos (imagens, favicon)
├─ src/
│  ├─ assets/css/         # Estilos globais (app.css, krefasy-theme.css, tailwind)
│  ├─ components/
│  │  ├─ layout/          # Shell: Header, Sidebar, Footer, PageHeader
│  │  │  └─ icon/         # Ícones SVG como componentes Vue
│  │  └─ composables/     # Lógica reutilizável (ex: use-chat-notifications)
│  └─ layouts/
│     ├─ app-layout.vue    # Layout autenticado (sidebar + header)
│     └─ auth-layout.vue   # Layout de login
│  └─ router/
│     └─ index.ts          # Definição de todas as rotas
│  └─ services/            # Camada de API (um ficheiro por domínio)
│  └─ stores/
│     ├─ index.ts          # Store de UI (tema, sidebar, i18n)
│     └─ krefasy.store.ts  # Store de negócio (dados da API)
│  └─ utils/               # Funções auxiliares
│  └─ views/               # Páginas da aplicação
│     ├─ auth/             # Login Krefasy
│     ├─ dashboard/        # Dashboard principal
│     ├─ customers/        # CRUD de clientes
│     ├─ loans/            # Empréstimos
│     ├─ parcels/          # Parcelas
│     ├─ chat/             # Chat / atendimento
│     ├─ users/            # Utilizadores admin
│     ├─ reports/          # Relatórios
│     └─ [configurações]/  # Moedas, países, produtos, etc.
│  └─ App.vue              # Root component
│  └─ main.ts              # Entry point
├─ docs/                  # Documentação (este ficheiro)
├─ env.example             # Variáveis de ambiente de referência
├─ Dockerfile              # Build para produção
├─ package.json
├─ tailwind.config.cjs
└─ vite.config.ts
```

Nota sobre views legadas

O projeto herda ~70 rotas de demonstração do template **Vristo** (`/apps/*` , `/elements/*` , `/forms/*` , etc.). Estas rotas **não fazem parte do produto Krefasy** e não aparecem no menu lateral. O menu ativo está definido em `Sidebar.vue` e aponta apenas para rotas Krefasy.

4. Arquitetura em camadas

Views (pages)	krefasy-chat.vue, etc.
Components	PageHeader, Modals
Stores (Pinia)	krefasy.store.ts
Services	messages.service.ts
API base	api.ts (Axios + JWT)
Backend .NET	credit-api.krefasy.com

Fluxo típico de dados

1. A **view** chama uma action do **store** ou diretamente um **service**
2. O **service** faz HTTP via `api.ts` (com token JWT automático)
3. A resposta segue o envelope `{ succeeded, data, message, errors }`
4. O **store** atualiza refs reativas; a view re-renderiza

Convenção de services

Cada domínio tem um ficheiro em `src/services/` :

Service	Domínio
<code>api.ts</code>	Cliente HTTP base, interceptors, tipos comuns
<code>auth.service.ts</code>	Login, logout, sessão
<code>customers.service.ts</code>	Clientes
<code>loans.service.ts</code>	Empréstimos
<code>parcels.service.ts</code>	Parcelas
<code>messages.service.ts</code>	Chat / conversas
<code>users.service.ts</code>	Utilizadores admin
<code>currencies.service.ts</code>	Moedas
<code>countries.service.ts</code>	Países
<code>loan-products.service.ts</code>	Produtos de empréstimo
<code>loan-status.service.ts</code>	Status de empréstimo
<code>interest-periods.service.ts</code>	Períodos de juros
<code>loan-interest-rates.service.ts</code>	Taxas de juros

5. Autenticação e segurança

Login

- Rota: `/auth/login` → `krefasy-login.vue`
- Endpoint: `POST /auth/login`
- Sessão guardada em `localStorage` com chave `USER_LOGIN` :

```
{
  "token": "jwt...",
  "user": { "id", "email", "name", "roles", "permissions" }
}
```

Proteção de rotas

Em `router/index.ts`, rotas com `meta.requiresAuth: true` verificam token no `beforeEach`. Sem token → redirect para `/auth/login`.

Interceptor Axios

- **Request:** adiciona `Authorization: Bearer {token}` automaticamente
- **Response 401:** limpa sessão e redireciona para login

Logout

`krefasyStore.logout()` → limpa `localStorage` + `resetStore()`

6. Integração com API (.NET)

URL base

`VITE_API_BASE_URL=https://credit-api.krefasy.com/api/v1`

Envelope de resposta

```
{
  "succeeded": true,
  "data": { ... },
  "message": "string",
  "description": null,
  "errors": null
}
```

Particularidade: PascalCase

O backend .NET usa **PascalCase** em query params e campos multipart. O frontend trabalha em camelCase e o service faz o mapeamento.

Exemplo — listar conversas:

Frontend (camelCase)	API (.NET PascalCase)
page	Page
limit	Limit
unreadOnly	UnreadOnly
clientId	ClientId

Exemplo — enviar mensagem (multipart):

Campo FormData	Valor
ConversationId	UUID da conversa
Content	Texto da mensagem
MessageType	TEXT, IMAGE OU FILE
Attachments[0]	Ficheiro

Normalização de respostas

`messages.service.ts` inclui `normalizeApiObject()` que converte chaves PascalCase da API para camelCase no frontend.

7. Páginas e rotas Krefasy (produto ativo)

Menu principal (Sidebar)

Rota	Página	Descrição
/dashboard	krefasy-dashboard.vue	KPIs, gráficos, resumo operacional
/customers	customers/index.vue	Listagem e CRUD de clientes
/customers/:id	customers/detail.vue	Detalhe do cliente
/loans	krefasy-loans.vue	Listagem de empréstimos
/loans/add	krefasy-loans-add.vue	Criar empréstimo
/loans/edit/:id	krefasy-loans-edit.vue	Editar empréstimo
/loans/view/:id	krefasy-loan-detail.vue	Detalhe completo do empréstimo
/loans/pending	krefasy-loans-pending.vue	Empréstimos pendentes de aprovação
/chat	krefasy-chat.vue	Chat com clientes (API real)
/users	users/index.vue	Gestão de utilizadores admin
/reports	krefasy-reports.vue	Relatórios

Configurações (submenu)

Rota	Página
/loan-products	Produtos de empréstimo
/loan-products/:id	Detalhe do produto
/currencies	Moedas
/countries	Países
/interest-periods	Períodos de juros
/loan-status	Status de empréstimos
/payment-method-types	Métodos de pagamento
/loan-interest-rates	Taxas de juros

Rotas auxiliares

Rota	Notas
/parcels	Parcelas (não no menu principal)
/parcels/overdue	Parcelas vencidas
/parcels/:id	Detalhe da parcela
/settings	Configurações gerais
/clients	Legado — usar /customers

8. Módulo de Chat (detalhe técnico)

Ficheiros principais

- src/views/chat/krefasy-chat.vue — UI principal
- src/views/chat/ConversationModal.vue — criar conversa (admin)
- src/services/messages.service.ts — API client
- src/composables/use-chat-notifications.ts — polling de não lidas
- src/stores/krefasy.store.ts — estado e actions

Endpoints utilizados

Método	Endpoint	Uso
GET	/conversations	Listar conversas (filtros server-side)
GET	/conversations/stats	Estatísticas (abertas, urgentes)
GET	/conversations/{id}	Detalhe
GET	/conversations/{id}/messages	Mensagens

POST	/conversations	Criar conversa
PUT	/conversations/{id}	Atualizar (ex: resolver)
PATCH	/conversations/{id}/mark-read	Marcar como lida
POST	/messages	Enviar mensagem (multipart)

Notificações globais

- Polling a cada **30 segundos** via `use-chat-notifications.ts`
- Badge na **Sidebar** (item Chat) com total de não lidas
- **Sino no Header** com dropdown das 5 conversas mais recentes
- Ao clicar numa notificação → navega para `/chat` e abre a conversa

9. Gestão de estado (Pinia)

useAppStore — UI / tema

- Tema claro/escuro/sistema
- Sidebar aberta/fechada
- Layout (app vs auth)
- Locale i18n

useKrefasyStore — negócio

Estado	Descrição
clients, loans, parcels	Listas de entidades
conversations, currentMessages	Chat
unreadCount, unreadConversations	Notificações de chat
dashboardStats	KPIs do dashboard
selectedClient, selectedLoan, etc.	Entidade em foco

Actions principais

```
// Exemplos de uso nas views
const store = useKrefasyStore();
await store.fetchClients();
await store.fetchLoans({ status: 'PENDING' });
await store.fetchConversations({ status: 'OPEN' });
await store.sendMessage({ conversationId, content, messageType: 'TEXT' });
await store.fetchUnreadChatNotifications();
```

10. Design system Krefasy

Definido em `src/assets/css/krefasy-theme.css` :

Token CSS	Valor	Uso
--krefasy-navy	#0e1133	Header, fundos escuros
--krefasy-purple	#801f82	Cor primária, links ativos
--krefasy-surface-muted	—	Fundo das páginas
--krefasy-text	—	Texto principal
--krefasy-text-muted	—	Subtítulos, breadcrumbs

Componentes de layout

- **PageHeader** — título, subtítulo, breadcrumbs e slot `actions` (botões)
- **Sidebar** — menu Krefasy only
- **Header** — título da página, sino de notificações, tema, menu utilizador

Padrão de páginas CRUD

1. `PageHeader` com título e botão "Novo"
2. Tabela com filtros
3. Modal para criar/editar (`*Modal.vue` na mesma pasta)
4. `SweetAlert2` para confirmações e erros

11. Variáveis de ambiente

Copiar `env.example` para `.env` :

Variável	Descrição
VITE_API_BASE_URL	URL base da API
VITE_API_TIMEOUT	Timeout HTTP (ms)
VITE_MAX_FILE_SIZE	Limite upload (bytes, default 10MB)
VITE_WEBSOCKET_URL	WebSocket (fase 2, não usado no MVP)
VITE_ENABLE MOCK_DATA	<code>false</code> em produção

Importante: variáveis `VITE_*` são expostas no *bundle*. Nunca colocar secrets no `.env` .

12. Setup local

Pré-requisitos

- Node.js 18+
- Yarn 1.x

Comandos

```
# Instalar dependências
yarn install
```

```
# Copiar variáveis de ambiente
cp env.example .env

# Servidor de desenvolvimento (http://localhost:5173)
yarn dev

# Build de produção
yarn build

# Preview do build
yarn preview

# Docker
yarn docker:build
```

Primeiro login

1. `yarn dev`
2. Abrir `http://localhost:5173`
3. Login em `/auth/login` com credenciais Admin da API

13. Build e deploy

Build

```
yarn build
# Output: dist/
```

O build corre `vue-tsc --noEmit` (verificação de tipos) seguido de `vite build`.

Docker

```
yarn docker:build
# ou
docker build -t backoffice-krefasy:latest .
```

O `Dockerfile` serve os ficheiros estáticos de `dist/` via nginx.

Push para registry

```
yarn docker:push:hub
# script: scripts/docker-push-hub.sh
```

14. Convenções para novos desenvolvedores

Nomenclatura

Tipo	Convenção	Exemplo
Views Krefasy	krefasy-*.vue OU index.vue	krefasy-loans.vue
Services	*.service.ts	loans.service.ts
Modals	*Modal.vue	CustomerModal.vue
Composables	use-*.ts	use-chat-notifications.ts

TypeScript

- Usar interfaces nos services (não `any` em código novo)
- Props tipadas com `defineProps<>()`
- Preferir Composition API + `<script setup>`

API

- Toda chamada HTTP passa por `src/services/`
- Nunca chamar Axios diretamente nas views
- Tratar erros com SweetAlert2 nas views; services lançam exceções

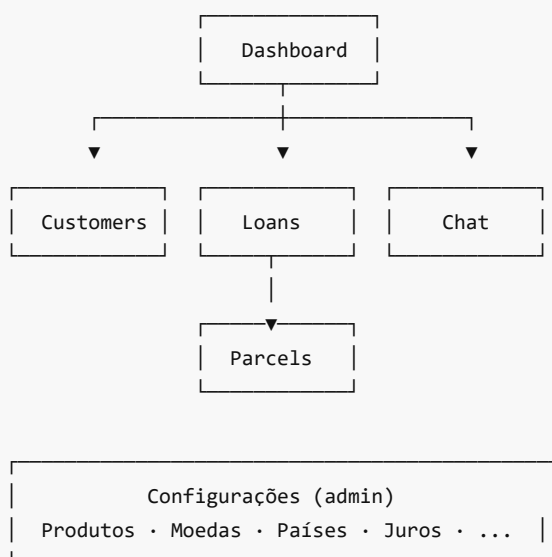
Estilo

- Tailwind + classes do tema Vistro (`.panel` , `.btn-primary` , `.form-input`)
- Tokens Krefasy via CSS variables (`var(--krefasy-purple)`)
- Componente `PageHeader` em todas as páginas novas

Git

- Não commitar `.env` (contém config local)
- Commits focados no "porquê", não só no "o quê"

15. Diagrama de módulos ativos



16. Contactos e recursos

Recurso	Localização
API Swagger	https://credit-api.krefasy.com (pedir acesso à equipa backend)
README do projeto	README-KREFASY.md
Variáveis de ambiente	env.example
Este documento	docs/ONBOARDING-DESENVOLVEDORES.md

Documento gerado para onboarding de desenvolvedores — Krefasy Backoffice.